Checkpoint 1 • Apresentação da Ideia

GPU-Friendly Laplacian Texture Blending

Investigação técnica e avaliação experimental de mesclagem piramidal de texturas em tempo real sem pré-computação.

Artigo de Referência

Bartlomiej Wronski

NVIDIA • JCGT (2025)

Área Temática

Área 3

Antialiasing, Texturas e Aparência

Integrantes da Equipe

Icaro Vinicius • Vinicius Macedo • Caio Dias

Prof. Wagner Barros

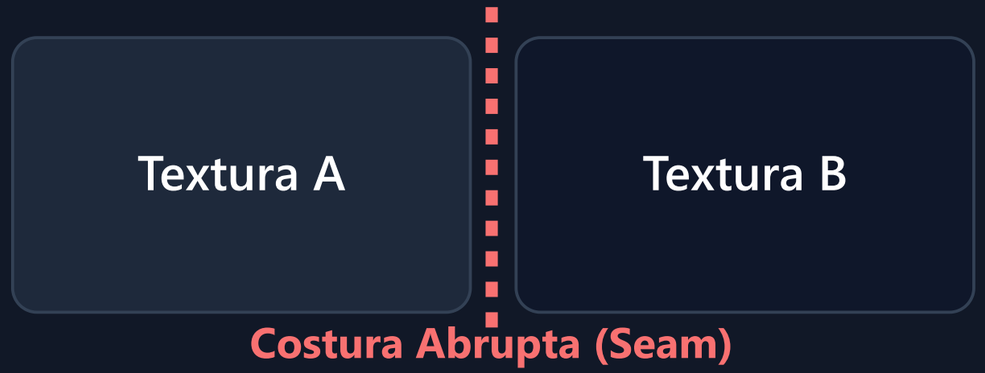
Pergunta 1 • O Problema em Computação Gráfica

O Dilema da Interpolação Linear de Texturas (Lerp)

Ao combinar materiais e texturas em tempo real, métodos pontuais impõem um compromisso visual destrutivo.

Transição Estreita (Narrow)

Artefato: Seams



- Gera **descontinuidades geométricas e visuais** imediatas.
- Quebra a continuidade natural de terrenos e materiais compostos.

Transição Ampla (Wide)

Artefato: Ghosting

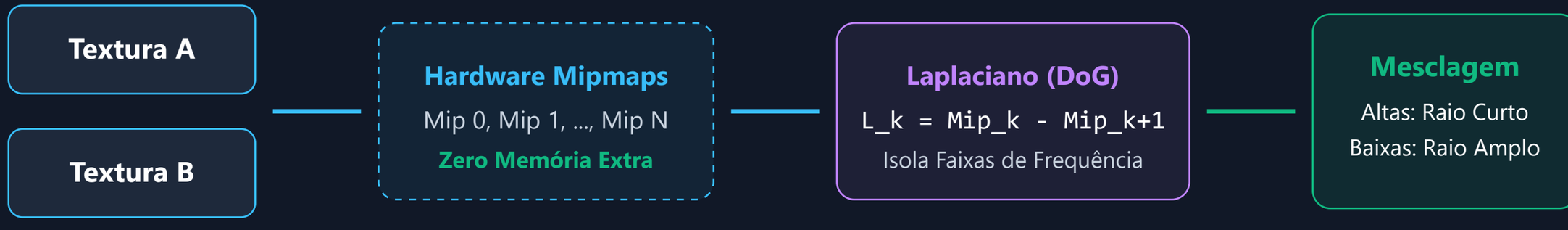


- **Atenuação severa de altas frequências:** detalhes finos desaparecem.
- Em mapas de normais, os relevos colapsam em direção ao vetor neutro (0,0,1).

Pergunta 2 • Contribuição Central

A Solução: Pirâmide Laplaciana nos Mipmaps de Hardware

Decomposição espectral aproximada diretamente no fragment shader sem pré-computação.



Alta Frequência

Microdetalhes mesclados com raio mínimo: **zero desfoque e sem ghosting**.

Baixa Frequência

Gradientes de cor média mesclados suavemente: **zero costuras abruptas**.

Tempo Real Puro

Construído inline na GPU: **compatível com materiais dinâmicos (Uber-Materials)**.

Implementação • Fragment Shader

Anatomia do Shader em ~15 Linhas de GLSL

```glsl
// 1. Amostragem dos Mipmaps (Hardware Bilinear)
for (int i = 0; i <= NUM_LEVELS; i++) {
  tex0_levels[i] = texture2D(tex0, uv, float(i));
  tex1_levels[i] = texture2D(tex1, uv, float(i));
  mask_levels[i] = texture2D(mask, uv, float(i));
}
```

```glsl
// 2. Acúmulo das Bandas Laplacianas (DoG)
for (int i = 0; i < NUM_LEVELS; i++) {
  vec4 l0 = tex0_levels[i] - tex0_levels[i+1];
  vec4 l1 = tex1_levels[i] - tex1_levels[i+1];
  blended += l0 * (1.0 - mask_levels[i]) + l1 * mask_levels[i];
}
```

```glsl
// 3. Resíduo Gaussiano Residual
blended += tex0_levels[N] * (1.0 - mask_levels[N]) +
           tex1_levels[N] * mask_levels[N];
```

Fase 1: Mipmap Sampling

Realiza N+1 texture fetches. As texturas de níveis inferiores possuem resolução drasticamente reduzida (1/4, 1/16, 1/64), apresentando altíssima localidade espacial e taxa de acerto nas caches L1/L2 da GPU.

Overhead Medido no Artigo (4K) **+0.087 ms**

Testado em NVIDIA RTX 4090 a 3840×2160 com 4 níveis Laplacianos. Custo imperceptível na taxa de quadros.

Pergunta 3 • Limitação e Oportunidade

Limitação Técnica: Sobrecarga de Texturas & Oportunidade

Em materiais realistas modernos ou celulares, a quantidade de leituras de textura pode sobrecarregar a placa de vídeo.

O Gargalo nas Unidades de Textura (TMUs)

• **Mistura Tradicional** (*Naive Blending*) = apenas **2 leituras** (1 de cada textura).
• **Mistura Laplaciana** (*Laplacian Blending* com 4 níveis) = salta para **10 leituras** (5 de cada textura).
• **Material Realista Completo** (*PBR*: Cor + Relevo + Brilho) = chega a **15 até 30 leituras** por ponto na tela (*fragment*)!

Em placas de vídeo integradas ou celulares (*mobile GPUs*), esse excesso de viagens à memória reduz a taxa de quadros (*FPS*).

A Oportunidade: Salto de Níveis (Level Skipping)

O autor sugere pular níveis alternados da pirâmide (Seção 6.2):

```
L̂_k ≈ Mip[k] - Mip[k+2]
```

• Reduz os acessos para apenas **(N/2) + 1** leituras de textura (*texture taps*), economizando quase 50%.
• *Lacuna Científica:* O artigo original apenas cita a ideia, mas **não mediu** a perda de qualidade e fidelidade visual (*SSIM / PSNR*).

Pergunta 4 • Proposta de Investigação

Proposta: Benchmark de Eficiência vs. Fidelidade

Investigar o compromisso entre redução de custo e fidelidade visual em engines gráficas.

Pergunta Central de Pesquisa

"Qual é o impacto quantitativo do Level Skipping e de diferentes filtros de interpolação na taxa de retenção de contraste e na similaridade estrutural (SSIM)?"

Eixo 1

Level Skipping

Avaliar pulos de 1 vs 2 níveis em diferentes resoluções.

Eixo 2

Filtros de Mipmap

Box filter padrão vs Lanczos / Bicúbico (análise de ringing).



Perguntas 5 e 6 • Metodologia e Baseline

Metodologia Experimental e Baselines

Protocolo rigoroso e reprodutível para comparação objetiva e visual.

1. Baselines

- **Linear Estreito:** Transição abrupta (evidencia costuras).
- **Linear Suave:** Transição ampla (evidencia ghosting).
- **Laplacian Full (6 níveis):** Referência máxima de qualidade.

2. Casos de Teste

- **Ruídos & Alta Frequência:** Padrões estocásticos celulares.
- **Texturas Fotográficas:** Rochas, cascalho, grama e areia.
- **Materiais Compostos:** Transições de solo, terra batida e rocha.

3. Métricas Objetivas

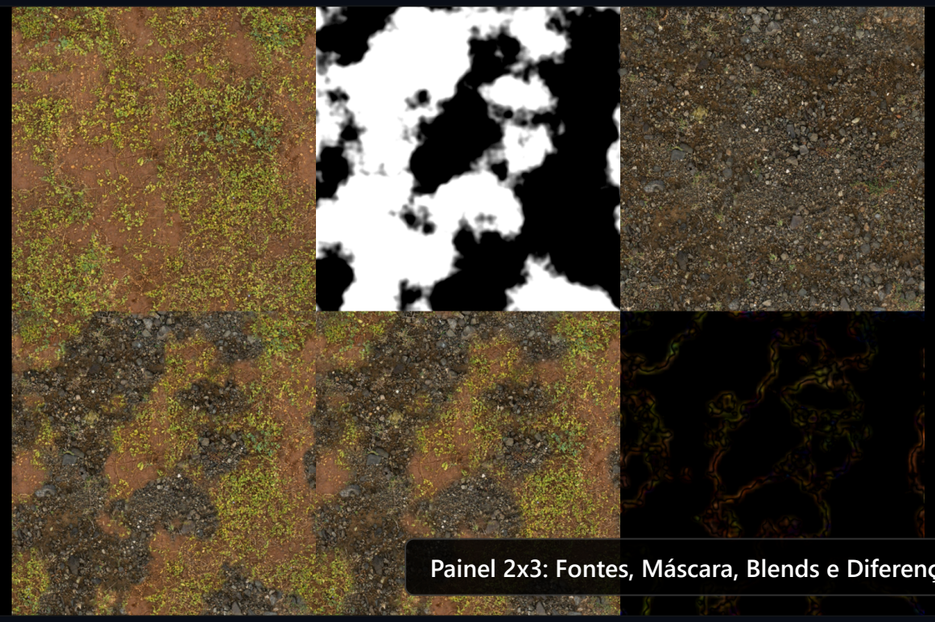
- **Variância Local (Var):** Retenção de contraste na borda.
- **SSIM & PSNR:** Fidelidade estrutural vs referência.
- **Frame-time (ms):** Custo computacional e saturação de TMU.

Pergunta 7 • Regra de Viabilidade Inicial

Viabilidade Inicial Comprovada no Ambiente

Implementação e resultados reais gerados no ambiente de desenvolvimento Linux da equipe:

Grid Comparativo | Baseline Linear | Laplacian (4 Níveis)



Painel 2x3: Fontes, Máscara, Blends e Diferença Residual

Métrica Obtida na Transição

98.5%

Retenção de variância local na borda, eliminando o efeito fantasma verificado na baseline linear.

Evidências do Edital Cumpridas

- **Demo WebGL oficial:** Executável em `demo_webgl/` com Three.js.
- **Pipeline Python:** Implementado em `src/laplacian_blend.py`.
- **Infraestrutura:** 100% pronta para a fase de testes de Level Skipping e LOD Adaptativo.

Proposta de Melhoria • Extensão para Cenas 3D

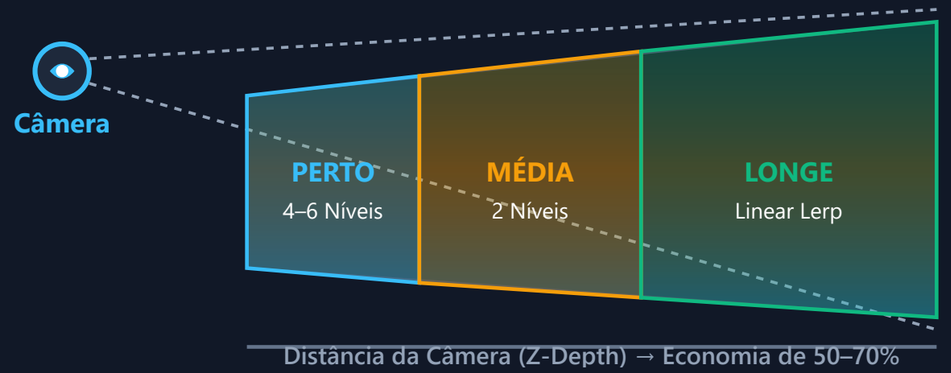
Proposta de Melhoria: LOD-Adaptive Blending

Ajuste dinâmico da pirâmide Laplaciana por distância em perspectiva via derivadas de hardware (dFdx/dFdy).

O Desperdício no Horizonte (3D)

Gargalo de TMU

- O artigo original calcula 4 a 6 níveis de forma **uniforme em todos os pixels**.
- Em cenas 3D com perspectiva, texels distantes já caem em mips grosseiros ($Mip \ge 3$).
- Calcular $L_0 = Mip_0 - Mip_1$ a 50m é **desperdício puro de TMU**, pois essas altas frequências são descartadas pelo rasterizador.



Engenharia do Shader (GLSL)

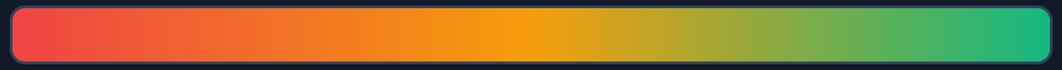
Hardware Nativo

A GPU calcula o LOD em tempo real via dFdx/dFdy, truncando o laço DoG:

```
// Quantidade de bandas varia com a distância
int levels = clamp(MAX_LEVELS - int(lod), 0, MAX_LEVELS);
for (int i = 0; i < levels; i++) {
  accum += (mips0[i] - mips0[i+1]) * mm[i+1] + ...;
}
```

Auditoria: Debug Heatmap na Demo WebGL

Checkpoint 2



4–6 Níveis **2 Níveis** **0 Extras (Lerp)**

Permite auditar ao vivo na cena 3D a transição suave de custo computacional sem qualquer salto visual (*popping*).

Conclusão

Perguntas & Discussão

Proposta tecnicamente fundamentada, viabilidade inicial comprovada e relevância prática imediata.

Artigo Base

GPU-Friendly Laplacian Texture Blending (Wronski, JCGT 2025).

Investigação & Extensão

Level Skipping e LOD-Adaptive Blending 3D (Desempenho vs Qualidade).

Status Técnico

Ambiente totalmente operacional e baseline reproduzida com sucesso.

Obrigado pela atenção!